LocatorX

Complete Application Documentation

Verified, ready-to-use test-automation locators for any webpage

Version 2.1 · June 2026

Contents

1. Overview 4

1.1 Who This Document Is For . . . 4

1.2 Why It Exists . . . 4

2. Key Features 5

3. Architecture & Tech Stack 6

3.1 Why Playwright, Not Cheerio / Axios . . . 6

3.2 Project File Layout . . . 6

4. Local Setup 8

4.1 Requirements . . . 8

4.2 Installation . . . 8

4.3 First-Run Checklist . . . 8

5. Using the Application (End-User Guide) 9

5.1 The Configure Panel . . . 9

5.2 Running an Analysis . . . 9

5.3 Reading the Results . . . 9

5.4 Exporting Results . . . 10

6. The Locator Generation Pipeline (Technical Detail) 11

Step 0 — SSRF Guard (before any navigation) . . . 11

Step 1 — Playwright Fetch . . . 11

Step 2 — Visible Element Extraction . . . 11

Step 3 — Mistral AI Locator Generation . . . 12

Step 4 — Live Locator Verification (multi-viewport) . . . 13

Step 5 — POM Generation (optional) . . . 13

6.6 Performance Optimizations . . . 13

6.7 Live Progress & Cancellation . . . 14

7. Login-Protected Pages, Sessions & Security 15

7.1 Security Hardening Overview . . . 15

7.2 One-Time Setup (per site) . . . 15

7.3 Every Analysis After That . . . 15

7.4 Session Expiry . . . 16

7.5 Managing Saved Sessions . . . 16

7.6 Session API Endpoints . . . 16

7.7 Environment Variables . . . 17

8. API Reference **18**

8.1 POST /api/analyze . . . 18

8.2 Progress & Cancellation Endpoints . . . 19

8.3 Session Endpoints . . . 19

9. Locator Confidence & Rejection Handling **20**

9.1 Confidence Levels . . . 20

9.2 Rejection Tags . . . 20

10. Output: Example Generated POM **21**

11. Export Formats **23**

11.1 Excel Export (.xlsx) . . . 23

11.2 JSON Export . . . 23

12. Tests **24**

13. Dependencies **25**

14. Glossary **26**

1. Overview

LocatorX is a self-hosted web application that automatically generates **verified, ready-to-use** test-automation locators (XPath, CSS Selector, and Playwright built-in locators) for any webpage — including pages that require a login.

Paste a URL, supply a free Mistral AI API key, and the agent will:

- Open the page in a real Chromium browser (via Playwright).
- Extract only the elements a human user can actually see and interact with.
- Ask Mistral AI to propose locators for each element.
- Run every proposed locator live against the page — at three screen sizes — to confirm it actually resolves to exactly one element.
- Return a clean, filtered table of verified locators, plus an auto-generated Page Object Model (POM) class — in either **Playwright (JavaScript)** or **Java (Selenium + PageFactory)** — ready to drop into a test suite.

The result is a list of locators that have already been proven to work — not a guess, not a static HTML scrape, but a locator that was tested against the real, fully-rendered page at the moment of generation.

1.1 Who This Document Is For

This documentation is written for two audiences at once:

- **End users / QA testers** who want to run the tool, paste URLs, and use the generated locators — see Sections 4, 5, and 6.
- **Developers / maintainers** who need to understand the internals, the API surface, the security model, and the pipeline logic in order to extend or debug the application — see Sections 3, 6, 7, 8, 9, and 12.

1.2 Why It Exists

Manually writing locators for an automation suite is slow and error-prone: testers guess at attributes, the locator might match zero or several elements, and nobody finds out until the test runs (and fails) later. This agent removes the guesswork by verifying every single locator against the live, rendered page — at desktop, tablet, and mobile widths — before it is ever shown to the user.

2. Key Features

- **Login-protected page support** — uses Playwright's `storageState` session pattern so the agent can generate locators for pages behind authentication, without ever seeing or storing a password.
- **Three locator types** — XPath, CSS Selector, and Playwright's own semantic locators (`getByRole`, `getByLabel`, etc.) — selectable independently.
- **Multi-viewport live verification** — every locator is executed against the real page at desktop (1280×800), tablet (768×1024), and mobile (375×667), now **in parallel** (one page per viewport in the same browser context). Only locators that resolve to exactly one element at *every* viewport are accepted; the rest are rejected and shown separately with a reason.
- **Live progress & one-click cancel** — the pipeline streams real stage-by-stage progress to the UI over Server-Sent Events, and any in-flight run can be aborted instantly with a Cancel button (the headless browser and pending AI call are torn down server-side).
- **Two POM output formats** — a ready-to-use **Playwright (JavaScript)** Page Object class, or a **Java (Selenium + PageFactory)** class using `@FindBy` annotations — both built from the same verified locator set.
- **Visibility-aware extraction** — runs entirely inside the browser using `getComputedStyle` and `getBoundingClientRect`, so hidden, zero-size, off-screen, or `aria-hidden` elements never pollute the results.
- **Rejection transparency** — every locator that fails verification is kept and tagged with a specific reason (`NO_MATCH`, `MULTI_MATCH`, `DUPLICATE`, `LOW_CONFIDENCE`, `VIEWPORT_DEPENDENT`) instead of being silently dropped.
- **Resilient AI calls** — parallel chunk dispatch, an adaptive inter-wave delay, exponential backoff on rate limits, and a rule-based fallback if Mistral AI fails entirely, so the pipeline never crashes outright.
- **Performance-tuned** — parallel multi-viewport verification and parallel Mistral chunking cut wall-clock time without changing the verification semantics or output, each with an automatic safe fallback (see Section 6.6).
- **Built-in security hardening** — SSRF protection on every URL, path-traversal guards on session names, request-body caps, locked-down CORS, and automatic cleanup of abandoned login browsers (see Section 7).
- **Anonymous Mode** — a one-click toggle that skips all saved-session storage and lookups entirely, for users who only ever test public pages.
- **Export options** — a formatted two-sheet Excel workbook (verified + rejected) or raw JSON.
- **Session auto-expiry** — saved login sessions self-delete two hours after they were captured, so stale credentials don't linger on disk indefinitely.

3. Architecture & Tech Stack

Layer	Technology	Why
Server	Node.js + Express	Lightweight, no build step. A thin `agent.js` entry wires up Express and two route modules; all logic lives in pure, independently-testable `lib/` modules.
Browser automation	Playwright (Chromium)	Real browser engine — correctly handles JavaScript-rendered apps (React, Angular, Vue), legacy Java UIs (JSF, Vaadin, Spring MVC), and self-signed SSL certs.
AI generation	Mistral AI (`mistral-small-latest`)	Free tier available, fast, supports native JSON-mode responses.
Frontend	Vanilla HTML / CSS / JavaScript	Zero build tooling — a single `index.html` is served directly by Express, no bundler needed.
Session storage	Local JSON files (Playwright `storageState`)	Cookies + localStorage + sessionStorage captured from a real authenticated browser session; git-ignored and auto-expiring.
Export	SheetJS (`xlsx`, vendored locally, CDN fallback)	Generates a formatted, multi-sheet Excel workbook entirely in the browser; the library is bundled at `public/vendor/` so export works offline.

3.1 Why Playwright, Not Cheerio / Axios

An earlier design fetched the raw HTML with `axios` and parsed it statically with `cheerio`. This breaks for any JavaScript-rendered application, because the HTML returned by the server is just the initial skeleton — the real content is injected by client-side JavaScript after the page loads. It also has no concept of computed CSS, so it cannot tell whether an element is actually visible.

The current design instead runs everything inside a real Chromium browser via Playwright. Element extraction happens through `page.$$eval()`, which executes JavaScript in the actual browser context and uses the same `getComputedStyle()` / `getBoundingClientRect()` APIs the browser itself uses to decide what a human can see. This is why the agent correctly handles dynamically-rendered single-page apps, lazy-loaded content, and hidden modals/carousels that a static HTML parser would get wrong.

> *Note: the legacy* `axios`, `cheerio`, *and* `puppeteer-core` *dependencies have been removed entirely —* `package.json` *now lists only the four packages the live pipeline actually uses (*`express`, `playwright`, `@mistralai/mistralai`, `cors`*).*

3.2 Project File Layout

`agent.js` is a thin entry point — it wires up Express, static file serving, and the two route modules, then starts listening. All real logic lives in `lib/` (pure functions, no HTTP awareness) and `routes/` (Express routers that call into `lib/`).

```
AI Locators/
|-- agent.js                Express app setup + listen (thin entry, no business logic)
|-- package.json            Dependencies (express, playwright, @mistralai/mistralai, cors)
|-- routes/
|   |-- analyze.js          POST /api/analyze + progress (SSE) + cancel endpoints
|   `-- sessions.js         Session endpoints (list/start/confirm/cancel/delete)
|-- lib/
|   |-- browser.js          Playwright: fetchAndExtract() + verifyAtViewports()
|   |-- mistral.js          Mistral call + retry/backoff + parallel chunking + prompts
|   |-- locators.js         Selector escaping, dedup, fallback, dynamic-ID check
|   |-- pom.js              POM generation (Playwright JS + Java Selenium)
|   |-- sessionStore.js     Session paths + guard, TTL, pending logins, active runs
|   |-- security.js         SSRF guard: scheme + private-address validation
|   `-- progress.js         Per-run SSE pub/sub for live pipeline progress
|-- test/                   Dependency-free node:test suites for the pure modules
|   |-- locators.test.js
|   |-- pom.test.js
|   `-- security.test.js
|-- public/
|   |-- index.html          Frontend UI (Anonymous Mode, live progress, cancel)
|   `-- vendor/
|       `-- xlsx.full.min.js  Vendored Excel-export library (offline; CDN fallback)
|-- sessions/               Saved <domain>.json (cookies/localStorage), gitignored, 2h TTL
|-- README.md               Architecture notes (developer-facing)
`-- USER_MANUAL.md          End-user / QA walkthrough of the UI
```

Each `lib/` module takes plain arguments and returns plain data, with no `req`/`res` objects — which is what makes it possible to unit-test the pipeline's logic (chunking, retries, verification, classification, POM output) without spinning up the full server.

4. Local Setup

> *Deployment to a public server is intentionally out of scope for this document — the application currently runs locally only. The reason is that the manual-login flow opens a visible (non-headless) Chromium window, which requires a real or virtual display; this rules out most free serverless hosting tiers.*

4.1 Requirements

- Node.js (any recent LTS version).
- A free Mistral AI API key — obtain one at `https://console.mistral.ai`.

4.2 Installation

```
npm install
npx playwright install chromium   # one-time download of the Chromium binary
node agent.js
```

Once running, open `http://localhost:3000`.

The server listens on port 3000 by default. Override it with the `PORT` environment variable (see Section 7.7).

4.3 First-Run Checklist

- Confirm the terminal prints the startup line ending in `running at http://localhost:3000`.
- Open the URL in a browser.
- Paste a target page URL and your Mistral API key.
- Leave Saved Session on “No session (anonymous)” unless the target page requires login (see Section 7).
- Choose your locator types and (optionally) a POM format, then click **Analyze Page**.

5. Using the Application (End-User Guide)

5.1 The Configure Panel

Field	Purpose
Page URL	The exact URL of the page you want locators for.
Mistral API Key	Your personal Mistral key — sent with each analyze request, never stored server-side.
Anonymous Mode	A toggle chip. When on, the app skips all saved-session storage and never calls the session endpoints — for users who only test public pages. The choice is remembered across reloads.
Saved Session	Optional dropdown of previously-captured login sessions (see Section 7). Leave on “No session” for public pages. Hidden while Anonymous Mode is on.
Locator Types	Toggle chips — choose any combination of Playwright Built-in, XPath, CSS Selector. At least one must remain selected.
POM Export	Toggle — when on, a Page Object class is generated alongside the locator table. A format dropdown selects **Playwright (JavaScript)** or **Selenium (Java)**.

5.2 Running an Analysis

Click **Analyze Page**. A pipeline indicator lights up each stage as it actually happens — the frontend opens a Server-Sent Events stream and the server pushes real progress events (not a fixed timer), so the bar reflects true pipeline state:

- Playwright Fetch
- Extract Elements
- Mistral AI
- Verify Locators (runs at all three viewports, in parallel)
- Build POM (skipped visually if the POM toggle is off)

A live timer next to the buttons shows elapsed time. Two more buttons sit alongside: **Cancel** (visible only while a run is in flight) aborts immediately — the browser’s request is aborted and the server tears down the headless browser and pending AI call — and **Clear** resets the form and results to start fresh.

Typical runs take anywhere from a few seconds (small page, Playwright-type locators only) up to a minute or more for large pages requesting all three locator types, since each chunk of elements is a separate AI call and verification runs at every viewport.

5.3 Reading the Results

Once complete, three stat cards appear:

- **Elements Found** — total visible, interactable elements extracted.
- **AI Generated** — total locator candidates Mistral AI proposed.
- **Verified** — locators that passed live verification at every viewport and are shown in the main table.

Below the stats, three tabs are available:

- **Locators Table** — every verified locator, with a per-row copy button for each selected locator type, the strategy used (ID-based, Attribute-based, Text-based, etc.), and a “Verified” badge. A search box

filters by element name or locator text.

- **Rejected** (with a live count badge) — every locator that failed verification, grouped by rejection reason. See Section 9 for what each tag means.
- **Playwright POM / Java POM** — the generated Page Object class, with Download and Copy buttons (the tab label and file extension follow the chosen format). Disabled if the POM toggle was off for that run.

5.4 Exporting Results

From the Locators Table tab:

- **Export Excel (.xlsx)** — downloads a two-sheet workbook: verified locators on Sheet 1, rejected locators (with full reasons) on Sheet 2. Both sheets have a frozen header row and first column, and pre-sized columns so long XPath/CSS strings are readable without manual resizing.
- **Export JSON** — downloads the raw verified-locators array as `locators.json`.

*Auto-restore: the last completed result is persisted in the browser, so an accidental refresh or reload restores the finished analysis instead of losing it. **Clear** drops that snapshot too.*

6. The Locator Generation Pipeline (Technical Detail)

This section describes what happens internally on every `POST /api/analyze` request — useful for developers debugging unexpected output or extending the pipeline. The route lives in `routes/analyze.js` and delegates to the `lib/` modules.

Step 0 — SSRF Guard (before any navigation)

Before a browser is even launched, the requested URL is passed through `assertSafeUrl()` (`lib/security.js`). Non-`http(s)` schemes are rejected, and the hostname is resolved so every resulting address can be checked against private/loopback/link-local ranges. Unsafe URLs are refused with HTTP 400/403 before any request is made (see Section 7.1).

Step 1 — Playwright Fetch

- Launches a headless Chromium browser with a realistic desktop User-Agent and a 1280×800 viewport (the first verification viewport).
- `ignoreHTTPSErrors: true` so self-signed / legacy SSL certificates don't block navigation.
- If a saved session was selected, it's attached to the new browser context via `storageState` before navigation.
- Navigates to the target URL (`waitUntil: 'domcontentloaded'`, 15s timeout).
- **Session validity check** (only when a session is in use): after navigating, the agent treats the session as expired only on a strong signal — either it was *redirected* to a login-looking URL (`/login`, `/signin`, `/auth`, `/sso`), or it landed on a login-looking URL that shows a password form when a login page was not itself requested. A password field alone is not enough (change-password and settings pages have one on a valid session). On a match it aborts with `SESSION_EXPIRED` rather than wasting an AI call on a login screen.
- **Smart wait:** polls every 150ms (up to 15s) until at least 3 candidate elements (`a, button, input, select, textarea, [role=button], [data-testid]`) are present — this is what lets the agent work on slow-rendering single-page apps.
- Additionally waits for `networkidle` (up to 2s) so lazy-loaded / XHR-driven content has a chance to finish.

Step 2 — Visible Element Extraction

Runs inside the browser via `page.$$eval()` against this selector:

```
a, button, input, select, textarea, label,
[role="button"], [role="link"], [role="checkbox"], [role="radio"],
[role="tab"], [role="menuitem"], [role="textbox"],
[data-testid], [data-cy], [name], h1, h2, h3
```

For each matched node, the following exclusion rules are applied, in order:

#	Rule	Reason
1	`!el.isConnected`	Element was detached from the DOM after matching.
2	`input[type="hidden"]`	Never visible to a user, never testable.
3	`aria-hidden="true"`	Explicitly marked as irrelevant to assistive tech / users.

#	Rule	Reason
4	Tag is `SCRIPT`, `STYLE`, `BASE`, or `NOSCRIPT`	Non-visual document elements that only matched because the selector includes `[name]`.
5	Class contains `sr-only`, `visually-hidden`, or `screen-reader-only`	Common accessibility-helper classes with no real visual presence.
6	`display:none` AND no useful attribute AND zero width/height	Genuinely invisible with nothing to anchor a locator to.

Elements inside dynamic containers (modals, dropdown menus, tooltips) that are currently hidden but *do* have a useful attribute (`id`, `name`, `aria-label`, `placeholder`, `data-testid`, `role`, or `href`) are deliberately kept — testers commonly need locators for elements that only become visible after an interaction.

After exclusion, each element is deduplicated by a fingerprint of `tag + id + name + first-40-chars-of-text + placeholder + aria-label`, and any `id` matching a known dynamic-ID pattern (`ember123`, `gwt-uid-4`, `:r0:`, long hex strings, pure numeric IDs, etc.) is stripped so the AI step never anchors a locator to a value that changes on every page load.

Step 3 — Mistral AI Locator Generation

- Elements are sent in **chunks of 25** to avoid oversized prompts and keep each AI call fast. Each element carries a per-chunk `ref` the model must echo back, so every returned locator is mapped to its exact source element by `ref` — not by array position. If the model drops, reorders, or merges items, locators are never silently attached to the wrong element, and anything omitted is rule-based backfilled.
- The prompt embeds a strict priority order: `data-testid/data-cy` → stable semantic ID → `name/aria-label/placeholder` → short visible text → positional index (last resort).
- Type-specific rules are injected only for the locator types the user selected, each with explicit do/don't examples (e.g. never use `@class` or class-based CSS selectors, since classes are treated as dynamic and unstable).
- Mistral is called with `responseFormat: { type: 'json_object' }` for reliable structured output.
- **Parallel dispatch:** chunks are sent in parallel *waves* (default 3, set via `MISTRAL_CONCURRENCY`) rather than strictly one at a time, while output is reassembled in the original element order.
- **Rate-limit handling:** on a 429 response, the call retries up to 3 times with exponential backoff (waits 10s, then 20s between attempts).
- **Adaptive inter-wave delay:** the pause *between* waves starts at 0ms and only grows (in 2s steps, capped at 10s) the moment any chunk in a wave is rate-limited — at which point the driver also drops to serial (concurrency 1) for the remainder to recover gracefully — decaying back toward 0ms as waves succeed cleanly. A run that is never rate-limited pays no per-wave tax.
- **Fallback:** if Mistral fails entirely (after retries, or returns unparseable JSON), the agent falls back to a deterministic, rule-based locator built directly from the element's own attributes (ID → data-testid → name → aria-label → placeholder → text → positional index), so the pipeline never crashes.

Post-processing on every AI response: XPath strings have any `[@class=...]` predicates stripped and `text()` comparisons rewritten to use `normalize-space()`; any CSS selector consisting purely of class names is replaced with a tag-based fallback (downgraded to Low confidence); and duplicate locator strings get an index suffix (`(xpath)[n]` / `:nth-of-type(n)`).

Step 4 — Live Locator Verification (multi-viewport)

Every candidate locator (excluding ones already flagged Low confidence) is executed against the live page from Step 1 — **at three viewports**: desktop (1280×800), tablet (768×1024), and mobile (375×667). `verifyAtViewports()` runs the three **in parallel**, giving each its own page in the *same* browser context (so a saved login session still applies), so the per-viewport resize + network-settle waits overlap instead of stacking. Within a viewport, locators are verified in concurrent batches of 8, and a cancel check runs at each batch boundary so an aborted run stops promptly. For each locator:

- **XPath** must resolve to exactly 1 element.
- **CSS Selector** must resolve to exactly 1 element.
- **Playwright locator** (`getByRole`, `getByLabel`, `getByPlaceholder`, `getByTestId`, `getByText`, `getByAltText`, `getByTitle`, or a raw `locator()`) is parsed back into a real Playwright locator by an eval-free parser and counted — 1 or more matches is accepted, since `getByText` can legitimately be a little more lenient.

A locator passes overall only if at least one selected type passes its check at **every** viewport. One that passes at some viewports but not others is tagged `VIEWPORT_DEPENDENT` instead of being misclassified as `NO_MATCH` or `MULTI_MATCH` based on whichever single size happened to be tested — responsive frameworks often mount entirely different markup per breakpoint. Locators that match a previously-verified locator's exact string are rejected as `DUPLICATE` rather than re-verified.

Step 5 — POM Generation (optional)

If the POM toggle was enabled, `generatePOM()` builds a Page Object class from the *verified* locator list only, in the format chosen by the `pomFormat` request field:

- **Playwright (JavaScript)** — class name derived from the page's `<title>`; one `get` property per locator (one variant per selected type, e.g. `get username()`, `get usernameByCss()`, `get usernameByXPath()`); `navigate()` and `waitForPageLoad()` helpers always included.
- **Java (Selenium + PageFactory)** — one `@FindBy` field per locator (CSS preferred, XPath fallback). Playwright built-in locators have no Selenium equivalent, so any element with *only* a Playwright locator is skipped and noted in the file header.
- **Both formats:** action helper methods (`fill...()`, `click...()`, `select...()`) are auto-generated for the first 8 interactive elements, and duplicate property/field names are auto-suffixed (two "Submit" elements become `submit` / `submit2`).

6.6 Performance Optimizations

Several stages are tuned to cut wall-clock time *without* changing the verification semantics or the output. Each optimization has an automatic safe fallback, so it can never break a run:

- **Parallel multi-viewport verification** — the three viewports used to be verified sequentially (three full passes, each with its own resize + network-settle wait). `verifyAtViewports()` now runs them concurrently, one page per viewport in the same context, so the settle waits overlap. Measured ~1.9× faster on a 40-locator synthetic page; the gain is larger on real pages where network-settle waits dominate. If the parallel path can't be set up (e.g. a flaky extra navigation), it falls back to the original sequential single-page resize.
- **Parallel Mistral chunking** — chunks dispatch in parallel waves (default 3, `MISTRAL_CONCURRENCY`) instead of strictly serially, preserving output order. Adaptive backoff is retained: on any rate-limit the delay grows and the pipeline drops to serial for the remainder to recover.

- **Locally-vendored Excel library** — the `xlsx` export library is bundled at `public/vendor/xlsx.full.min.js` and loaded locally first (works offline and under strict CSP), falling back to the CDN only if the local copy is missing.

6.7 Live Progress & Cancellation

Each analyze run is identified by a short `runId` the client generates. It enables two cross-request features handled by `lib/progress.js` and the `activeRuns` registry in `lib/sessionStore.js`:

- **Server-Sent Events progress** — before POSTing `/api/analyze`, the client opens `GET /api/analyze/progress?runId=...`. The handler emits a real event as each stage completes, so the UI reflects actual pipeline state. The channel buffers and replays events to a late subscriber, so the inherent race between opening the stream and starting the run loses nothing.
- **Cancellation** — `POST /api/analyze/cancel` flags the run and force-closes its headless browser, which makes any in-flight Playwright call reject promptly; checkpoints between the expensive stages also abort it. Cancelled runs return `200 { cancelled: true }` rather than an error.
- **Leak safety** — orphaned runs are swept after 10 minutes, and any in-flight analyze browsers are also closed on graceful shutdown (`SIGINT` / `SIGTERM`).

7. Login-Protected Pages, Sessions & Security

For pages that require authentication, the agent uses Playwright's `storageState` pattern — capturing cookies, localStorage, and sessionStorage from a real, manually-completed login. **The agent never sees, transmits, or stores a password.**

7.1 Security Hardening Overview

Because the tool drives a real browser against URLs you supply and stores authenticated session state on disk, several hardening measures are built in:

- **SSRF protection** — both navigating endpoints (`/api/analyze` and `/api/session/start-login`) run the URL through `assertSafeUrl()` first. Only `http` / `https` schemes are allowed, and the hostname is resolved so requests to loopback (`127.0.0.0/8`, `::1`), private ranges, link-local (`169.254/16`, which includes the cloud metadata endpoint `169.254.169.254`), and `localhost` are refused with HTTP 403. Set `ALLOW_PRIVATE_TARGETS=1` to test locally-hosted apps.
- **Path-traversal protection** — client-supplied session names pass through `resolveSessionPath()`, which rejects anything that isn't a simple name (`^[A-Za-z0-9._-]+$`, no slashes, no `..`) and confirms the resolved file stays directly inside `sessions/`.
- **Resource-leak protection** — abandoned headed-login browsers *and* orphaned in-flight analyze runs are swept after 10 minutes, and all open browsers (login + analyze) are closed on graceful shutdown (`SIGINT` / `SIGTERM`) so no orphaned Chromium processes are left behind.
- **CORS locked down** — the SPA is same-origin, so no cross-origin access is granted by default. Set `ALLOWED_ORIGIN` to permit a specific origin.
- **Request body cap** — JSON bodies are limited to 256 KB.
- **Secrets never persisted** — the Mistral API key is supplied per request and is never written to disk or logged.

7.2 One-Time Setup (per site)

- Type the site's URL into the Page URL field.
- Click **Login & Save Session**.
- A **visible** Chrome window opens.
- Log in manually — username/password, 2FA, OTP, SSO redirect, CAPTCHA — exactly as you normally would. Because a human completes the actual login, the agent needs no custom code per login type.
- Click "I'm Logged In" in the UI.
- The agent captures the authenticated browser state and saves it to `sessions/<domain>.json` (e.g. `sessions/premium-emdha-sa.json`).

7.3 Every Analysis After That

- Enter *any* URL on that same site — not just the page you logged in from.
- Select the saved session from the Saved Session dropdown.
- The agent loads the saved cookies/storage into a fresh browser context and navigates straight to your target URL, already authenticated.
- Extraction, AI generation, and verification proceed exactly as for a public page.

The saved session is purely an authentication “key” unrelated to any specific page, so it works across the entire authenticated app once captured.

7.4 Session Expiry

Two independent expiry mechanisms protect against using a dead session:

- **Fixed 2-hour TTL (proactive).** Every saved session file auto-deletes 2 hours after it was last saved/refreshed, regardless of the site’s own cookie expiry. This is a fixed, non-configurable, agent-side rule, checked lazily (no background timer): `GET /api/sessions` prunes any file older than 2 hours before returning the list, and `POST /api/analyze` deletes an over-age session on the spot and fails with `code: SESSION_EXPIRED`.
- **Reactive detection.** The website’s own rules can invalidate a session before the 2-hour mark. The agent can’t predict this, so after navigating with a saved session it applies the strong-signal login-page check from Step 1 and, if triggered, stops immediately and surfaces a “session expired or invalid” banner with a one-click Re-login button.

7.5 Managing Saved Sessions

- **Switch sites** — pick a different session, or select “No session” for public pages.
- **Session age display** — each dropdown entry shows a human-readable “saved X ago” label, purely informational (not a validity guarantee).
- **Delete a session** — select it, then click the Delete button next to the dropdown.
- **Auto-delete** — every session also self-deletes after 2 hours; no manual cleanup required.
- **Re-login** — clicking **Login & Save Session** again overwrites the existing file for that domain and resets its 2-hour clock.

7.6 Session API Endpoints

Endpoint	Purpose
`GET /api/sessions`	Lists saved sessions with human-readable age labels (after pruning expired ones).
`POST /api/session/start-login`	Opens a visible browser window for manual login (URL is SSRF-checked first).
`POST /api/session/confirm-login`	Captures the authenticated state and writes `sessions/&lt;domain&gt;.json`.
`POST /api/session/cancel-login`	Closes the login browser without saving.
`DELETE /api/sessions/:domain`	Deletes a specific saved session file (name is path-traversal checked).

7.7 Environment Variables

Variable	Default	Purpose
`PORT`	`3000`	Port the server listens on.
`ALLOW_PRIVATE_TARGETS`	`unset`	Set to `1` to allow analyzing localhost / private-IP targets.
`ALLOWED_ORIGIN`	`unset`	Explicit CORS origin to allow (otherwise cross-origin is disabled).
`MISTRAL_CONCURRENCY`	`3`	How many locator-generation chunks to send to Mistral in parallel per wave.

> ***This is intended as a local developer tool.*** *It has no user authentication — do not expose it directly to untrusted networks. If you must, put it behind an authenticating reverse proxy and review the settings above.*

8. API Reference

All endpoints are served by the single Express process started via `node agent.js`, listening on port 3000 by default.

8.1 POST /api/analyze

The main pipeline endpoint. **Request body:**

```
{
  "url": "https://example.com/login",
  "mistralApiKey": "your-mistral-key",
  "locatorTypes": ["playwright", "xpath", "css"],
  "generatePom": true,
  "pomFormat": "playwright",
  "sessionDomain": null,
  "runId": "run_optional_id"
}
```

Field	Type	Required	Notes
`url`	string	yes	Target page URL (SSRF-validated).
`mistralApiKey`	string	yes	Your Mistral API key.
`locatorTypes`	array	no	Any combination of `'xpath'`, `'css'`, `'playwright'`; default `['playwright']`; at least one enforced.
`generatePom`	boolean	no	Whether to build a Page Object class (default `true`).
`pomFormat`	string	no	`'playwright'` (default) or `'java'` — selects the POM output template.
`sessionDomain`	string|null	no	Domain key of a previously-saved session, e.g. `"premium-emdha-sa"` (default `null`).
`runId`	string	no	Client-generated id (`[A-Za-z0-9_-]{1,64}`) tying this run to its progress stream and cancel request; auto-generated server-side if omitted (then it can't be cancelled from the client).

Success response (200):

```
{
  "url": "...",
  "pageTitle": "...",
  "totalExtracted": 42,
  "totalGenerated": 40,
  "totalVerified": 31,
  "totalRejected": 9,
  "rejectionSummary": {
    "LOW_CONFIDENCE": 2, "NO_MATCH": 4, "MULTI_MATCH": 2,
    "DUPLICATE": 1, "VIEWPORT_DEPENDENT": 0
  },
  "locatorTypes": ["playwright", "xpath", "css"],
  "generatePom": true,
  "sessionUsed": null,
  "locators": [ /* verified locator objects */ ],
  "rejected": [ /* rejected objects w/ rejectionTag + rejectionReason */ ],
  "pom": "// generated Page Object Model source..."
}
```

If the run is cancelled mid-flight (see Section 8.2), the endpoint instead returns `200 { "cancelled": true }` — a cancel is not treated as an error.

Error responses:

Status	Condition
400	Missing `url`/`mistralApiKey`, no locator type selected, invalid/unsafe URL, invalid or not-found session name, page fetch failed, or zero visible elements extracted.
401	Selected session is expired (`code: "SESSION_EXPIRED"`) — either by the 2-hour TTL or by reactive login-page detection.
403	URL points at a private/internal address or uses a disallowed scheme (`code: "UNSAFE_URL"`).
500	Unexpected server error during the pipeline.

8.2 Progress & Cancellation Endpoints

These share the `runId` sent to `/api/analyze` (see Section 6.7):

```
GET  /api/analyze/progress?runId=<id>     (Server-Sent Events stream)
     -> event data: { "key": "p1.."p5"|"end", "state": "active"|"done"|
                      "skip"|"cancelled"|"error", "msg"?, "count"?, ... }

POST /api/analyze/cancel        body { "runId" }
     -> { "success": true }     (idempotent; unknown/finished ids also succeed)
```

The progress stream is opened just before the analyze POST; its channel buffers and replays events so nothing emitted during the connect race is lost. Cancelling flags the run and force-closes its headless browser.

8.3 Session Endpoints

See Section 7.6 for the full table. Request / response shapes:

```
GET  /api/sessions
     -> { "sessions": [ { "domain", "filename", "savedAt", "ageLabel" } ] }

POST /api/session/start-login    body { "url" }
     -> { "token", "domain", "message" }

POST /api/session/confirm-login  body { "token" }
     -> { "success": true, "domain", "filename" }

POST /api/session/cancel-login   body { "token" }
     -> { "success": true }

DELETE /api/sessions/:domain
     -> { "success": true }
```

9. Locator Confidence & Rejection Handling

9.1 Confidence Levels

Level	Assigned When	Stability
High	`data-testid`/`data-cy`, a stable (non-dynamic) `id`, `name`, or `aria-label`	Tied to semantic, intentionally-set attributes — very stable.
Medium	`placeholder` or short visible text	Mostly stable — text content can change with copy edits.
Low	Positional index (e.g. `(//button)[3]`)	Breaks on any layout change — excluded from results by policy.

Only locators that are *not* Low confidence, and that pass live verification at every viewport, ever reach the output table.

9.2 Rejection Tags

Every locator that fails to qualify is kept (not discarded) and tagged with a specific reason, visible in the Rejected tab and in the Excel export's second sheet:

Tag	Meaning	What it usually indicates
`LOW_CONFIDENCE`	Index-based locator, excluded by policy before verification was even attempted.	Safe to ignore — an intentional exclusion, not a bug.
`NO_MATCH`	The locator matched 0 elements on the live page (at every tested viewport).	Mistral generated an incorrect attribute, or the page uses non-standard HTML.
`MULTI_MATCH`	The locator matched 2+ elements — ambiguous.	The page has several visually-similar elements; adding `data-testid` attributes in the app would help.
`DUPLICATE`	The exact same locator string was already verified for a different element.	The page has repeated structural blocks; an index-based locator may be the only remaining option.
`VIEWPORT_DEPENDENT`	Matched exactly 1 element at some viewports (desktop/tablet/mobile) but not all.	The page renders different markup per breakpoint; confirm your intended test viewport before using it.

Terminal logs also print a line per rejection (`x [TAG] elementLabel: reason`) for debugging during development.

10. Output: Example Generated POM

The same verified locator set produces either of the two templates below; only the output differs. **Playwright (JavaScript):**

```
// ============================================================
// Page Object Model - Auto-generated by LocatorX
// Page     : Login Page
// URL      : https://example.com/login
// Date     : 2026-06-24
// Locators : Playwright built-in, XPath, CSS
// Note     : Only HIGH confidence, visible & verified locators
// ============================================================

const { expect } = require('@playwright/test');

class LoginPagePage {
  constructor(page) {
    this.page = page;
    this.url  = 'https://example.com/login';
  }

  async navigate() { await this.page.goto(this.url); }

  // -- Locators -----------------------------------------------
  get username()        { return this.page.getByLabel('Username'); }
  get usernameByCss()   { return this.page.locator('[name="username"]'); }
  get usernameByXPath() { return this.page.locator('xpath=//input[@name="username"]'); }
  get password()        { return this.page.getByLabel('Password'); }
  get loginButton()     { return this.page.getByRole('button', { name: 'Sign In' }); }

  // -- Actions ------------------------------------------------
  async fillUsername(value) { await this.username.fill(value); }
  async fillPassword(value) { await this.password.fill(value); }
  async clickLoginButton()  { await this.loginButton.click(); }

  // -- Helpers ------------------------------------------------
  async waitForPageLoad() { await this.page.waitForLoadState('networkidle'); }
}

module.exports = { LoginPagePage };
```

Java (Selenium + PageFactory) — same locators, CSS preferred, XPath fallback:

```
// ============================================================
// Page Object Model - Auto-generated by LocatorX
// Page     : Login Page
// URL      : https://example.com/login
// Locators : Selenium WebDriver (CSS preferred, XPath fallback)
// Note     : Only HIGH confidence, visible & verified locators
// ============================================================

package com.locators.pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;
import org.openqa.selenium.support.ui.Select;

public class LoginPagePage {

    private final WebDriver driver;
    private final String url = "https://example.com/login";

    // input | Attribute-based | Confidence: High
    @FindBy(css = "[name=\"username\"]")
    private WebElement username;

    // button | Text-based | Confidence: High
    @FindBy(css = "button[type=\"submit\"]")
    private WebElement loginButton;

    public LoginPagePage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }

    public void navigate() { driver.get(url); }

    public void fillUsername(String value) {
        username.clear();
        username.sendKeys(value);
    }

    public void clickLoginButton() { loginButton.click(); }
}
```

11. Export Formats

11.1 Excel Export (.xlsx)

Generated entirely client-side via SheetJS (loaded on demand from a CDN). Produces a workbook with two sheets:

- **Sheet 1 — Verified Locators:** #, Element, Type, the selected locator-type columns, Strategy, Confidence, Status. Dark-green header, frozen header row and first column, pre-sized columns.
- **Sheet 2 — Rejected Locators:** the same base columns plus Rejection Tag and Reason. Dark-red header, same freeze/sizing behaviour.

11.2 JSON Export

Downloads the raw verified-locators array exactly as returned by `/api/analyze`, as `locators.json` — useful for piping into another tool or script.

12. Tests

The pure-logic `lib/` modules are covered by a dependency-free test suite using Node's built-in runner:

```
npm test        # runs: node --test
```

The suites (in `test/`) cover:

- **Selector escaping** — XPath `concat()` quote-switching and CSS identifier / attribute-value escaping (`locators.test.js`).
- **Locator de-duplication & immutability** — index-suffixing of duplicate strings without mutating the input array (`locators.test.js`).
- **Both POM output formats** — Playwright JavaScript and Java Selenium, including name de-duplication and Playwright-only skipping (`pom.test.js`).
- **SSRF address/scheme checks** — private/loopback/link-local detection for IPv4 and IPv6 (`security.test.js`).

Because each module takes plain arguments and returns plain data, these run without starting the server or opening a browser.

13. Dependencies

From `package.json`:

Package	Version	Role
`express`	`^5.2.1`	HTTP server and routing.
`playwright`	`^1.60.0`	All browser automation — fetch, extract, verify, headed login.
`@mistralai/mistralai`	`^2.2.5`	AI locator generation.
`cors`	`^2.8.6`	CORS middleware (locked down by default).

The earlier `axios`, `cheerio`, and `puppeteer-core` dependencies have been removed — the current pipeline runs exclusively on Playwright + Chromium.

Frontend-only dependency, loaded on demand from a CDN rather than installed via npm:

Library	Purpose
`SheetJS (xlsx)`	Builds the formatted two-sheet Excel export in the browser.

14. Glossary

Term	Meaning
Locator	A string that uniquely identifies one element on a page, used by test automation to find and interact with it (an XPath, a CSS selector, or a Playwright `getByRole(...)` call).
POM (Page Object Model)	A test-automation design pattern where each page has a corresponding class exposing its locators as properties/fields and its common interactions as methods.
`storageState`	Playwright's mechanism for capturing and replaying a browser's cookies, localStorage, and sessionStorage — used here to persist an authenticated session.
Verification	Running a generated locator against the live page (at every tested viewport) to confirm it resolves to exactly one element before showing it to the user.
Confidence	A High/Medium/Low rating describing how stable a locator's underlying attribute is expected to be over time.
Dynamic ID	An auto-generated element ID that changes between page loads (e.g. `ember123`, `:r0:`) and is therefore unsafe to use as a locator.
SSRF	Server-Side Request Forgery — tricking a server into making requests to internal addresses. Blocked here by `assertSafeUrl()`.
Viewport-dependent	A locator that matches exactly one element at some screen sizes but not others, because the page renders different markup per breakpoint.